A Distributed Multi-Agent AI Platform for Scalable Forensic Audit and Compliance

Podichetty Samiksha[1], Chandana Devanaboyina[1], Medapally Shiva Reddy[1] and Dr. B. Veera Jyothi[2]
[1]Department of Information Technology, Chaitanya Bharathi Institute of Technology, Hyderabad, India
Email: samiksha10.1b@gmail.com, chandanadev6189@gmail.com, shivashankarmedapalli@gmail.com
[2]Associate Professor, Department of Information Technology,
Chaitanya Bharathi Institute of Technology, Hyderabad, India
Email: veerajyothi_it@cbit.ac.in

***Abstract*—Procurement audit checks every invoice against its purchase order, goods receipt, vendor record and approval trail, and justifies each exception by a written policy. Large language models (LLMs) can read such policies, but they often find errors without citing the governing rule, and policy thresholds quietly drift away from the prompts and code meant to enforce them. We built a distributed multi-agent platform for forensic audit around a policy compiler, which turns the policy document into a typed clause registry; twelve deterministic controls apply its values before any LLM is called. No threshold lives in code or prompts, and no model can withdraw a control's finding. We also formalise threshold drift: an auditor with frozen thresholds errs on exactly the decisions a policy change moves, and the direction of the change fixes the type of error. On synthetic ledgers of 40,000 invoices, the compiler recovered 196 of 196 thresholds from fourteen documents and 280 of 280 from twenty held-out ones. The controls made no wrong decision in 1,360,000 audits, while frozen thresholds got up to 30.7% of invoices wrong. On three benchmark ledgers the controls reached an F1 of 0.833 at a precision of 1.000, against 0.536 for an isolation forest and 0.498 for a one-class SVM, at under 20 μs per invoice. On a 34-invoice sample, agents on a local 7B model found 12 of 20 judgement anomalies, but only 0.375 of their findings named the broken rule. All data are synthetic.**

***Index Terms*—forensic audit, multi-agent systems, large language models, policy as data, deterministic controls, threshold drift.**

I. Introduction

In a procurement audit, each invoice must match an approved purchase order (PO) and a goods receipt note (GRN), come from a genuine, tax-compliant vendor, and be approved within the approver's delegated limit. The evidence is spread over several records, and what counts as an exception is set by the organisation's own policy. Continuous auditing argues for testing every transaction instead of a sample [1].

Since the policy is written in prose, large language models (LLMs) seem well suited to this work. AuditBench [2] shows their limits: LLMs can spot errors in transaction histories, but explain them poorly and seldom cite the relevant standard. Systems of specialised LLM agents are also being proposed for financial tasks [3]. Building such a pipeline, we met a second, structural problem, the *policy–implementation gap*: limits were stated in prose, one was copied as a number into an agent prompt, and whether a rule was checked depended on an agent choosing to call a tool. Each policy revision meant editing the system by hand, with no evidence that the edit matched the document.

We describe a distributed multi-agent platform for forensic audit in procurement. It is *forensic* because every decision can be reconstructed: each finding carries its rule, its evidence and the policy page it relies on, and every stage writes to an audit trail. It is *distributed* because it is built from separate services that communicate over network interfaces. We keep the base pipeline of [2], [3] and add one feature that closes the gap. Our contributions are:

1. a *policy compiler* that turns the policy document into an executable clause registry and refuses, with reasons, any document it cannot execute (Subsection V-B);
2. a controls-first design with a *non-interference* guarantee: no LLM stage can remove or weaken a deterministic finding (Proposition 1);
3. a formal account of *threshold drift*: a frozen auditor errs on exactly the decisions a policy change displaces, with a polarity set by the direction of the change (Propositions 2 and 3), checked on 34 policy variants;
4. a comparison with two classical anomaly detectors and an end-to-end evaluation of the LLM stages, released as open, reproducible code and data.

II. Base Papers

A. AuditBench

Wang et al. [2] inject errors into synthetic transactions paired with financial tables and evaluate LLMs on financial-statement audit. They found that LLMs detect errors but explain them poorly and seldom cite the governing standard, which is why every finding in our system must name its rule. We adopt their construction of evaluation data with injected, labelled errors and their requirement that a finding cite its rule. AuditBench measures models; it does not propose a system that removes the weakness, nor does it cover procurement documents checked against an organisation's own policy.

B. LLM-Based Multi-Agent Systems in Finance

Liu et al. [3] survey LLM-based multi-agent systems for financial markets and argue that heterogeneous, autonomous and boundedly rational agents describe markets better than traditional models. We adopt their architecture of specialist agents, each with its own role and view of the data, whose outputs are combined. The survey concerns market simulation rather than audit, and lists regulatory traceability of agent decisions as an open problem, one that an audit system has to answer concretely.

III. Related Work

A. Anomaly Detection

Anomaly detection ranks records by how unusual they are [4], and statistical fraud detection has long been reviewed [5]. Isolation forests [6] and one-class support vector machines (SVMs) [7] are common unsupervised choices, and we use both as baselines. A detector returns a score, not a rule: an unusual record need not break the policy, a violation such as an invoice approved above a delegated limit need not look unusual, and every policy change means retraining.

B. LLMs and Agents in Audit

LLMs have been used to match financial text against audit requirements [8] and to check the regulatory compliance of financial reports [9]. Generated text can state facts the input does not support [10], which an audit cannot accept; retrieval-augmented generation [11] grounds answers in retrieved passages, but grounding alone does not make a rule citation correct. Surveys of LLM-based agents are given in [12], debate between agents improves factual accuracy [13], and human review of uncertain cases remains the accepted safeguard for high-stakes decisions [14].

C. Research Gap

Earlier work either detects anomalies without naming a rule or lets an LLM reason over the policy text. To our knowledge, no published system lets the policy document itself configure executable audit controls, so that the thresholds the system applies provably match the document.

IV. Problem Formulation

A ledger is a set $\mathcal{L} = \{x_1, \dots, x_N\}$ of transactions. Each transaction x is an invoice together with its PO, its GRN, the vendor master record, the approval-log entries and the other invoices of the same vendor.

Definition 1 (Policy)**.** *A policy version is a tuple* $P = (K, s, \kappa, \theta)$*: a set* K *of rule identifiers; a severity* $s : K \to \{\text{critical}, \text{high}, \text{medium}, \text{low}\}$*; a kind* $\kappa : K \to \{\text{control}, \text{judgement}\}$*, which says whether a rule can be decided by comparing fields; and parameters* $\theta = (\theta_k)_{k \in K}$*. We write* $K_c = \{k : \kappa(k) = \text{control}\}$.

Our reference policy (version 2.5) has $|K| = 16$ rules, $|K_c| = 12$, and 14 parameter values. Let $V_P(x) \subseteq K$ be the set of rules that x violates under P, as computed by an independent validator, and let $V_P^c(x) = V_P(x) \cap K_c$. An auditor $\mathcal{A}$ cites a set $C_{\mathcal{A}}(x)$ of rules.

Definition 2 (Adherence and Displacement)**.** *The adherence of* $\mathcal{A}$ *to* P *is*

$$\mathrm{A}_P(\mathcal{A}) = \frac{1}{N} \sum_{x \in \mathcal{L}} \mathbf{1}\big[C_{\mathcal{A}}(x) = V_P^c(x)\big]. \tag{1}$$

The decision displacement of a change from P_0 *to* P *is* $\Delta(P_0, P) = \{x \in \mathcal{L} : V_P^c(x) \neq V_{P_0}^c(x)\}$.

The *policy-driven* auditor runs the controls with the parameters $\hat{\theta} = E(D)$ that the compiler E extracts from the document D of P. The *frozen* auditor runs them with the parameters θ_0 of P_0, like a system whose thresholds sit in prompts and code until someone edits them.

V. Proposed System

A. Architecture

The platform consists of audit workers, a policy service, a relational data store (SQLite or PostgreSQL), a model endpoint with an OpenAI-compatible interface, and a representational state transfer (REST) application programming interface (API) with a web application for reviewers (Fig. 1). The unit of work is one invoice, whose transaction carries every record it needs, so workers need no coordination. The pipeline is a LangGraph [15] state graph: evidence extraction, deterministic controls, three specialist agents in parallel (math and approvals; tax and identity; vendor intelligence), a challenger, a confidence stage, risk scoring and routing, and human review with an audit pack. Each agent sees only the records its rules need, and code discards any finding that cites a rule outside the agent's remit. Results are appended, never overwritten.

B. Policy Compilation

The compiler (Algorithm 1) reads every text line of the portable document format (PDF) file with its font size, position and page, and splits the text into numbered rules by layout rather than keywords: the most common font size is the body size, a line at least one point larger is a heading, and a line beginning "Rule ⟨number⟩ (⟨severity⟩)," starts a rule. Each parameter has one pattern, matched against the wording of the rule that states it; approval limits are read from the table of roles. No language model is involved: a value is either found in its sentence or reported missing. The rule set K and the kinds κ come from a reference version, which records what the system can execute.

C. Deterministic Controls

A control $c_k(x; \theta_k)$ is a pure function of a transaction and the parameters of its rule. It returns nothing or a finding $f = (k, s(k), \mathcal{E})$, where $\mathcal{E}$ is the set of field values the finding rests on; when stored, the finding is linked to the document and page of its rule. A control has no constants of its own. For example, the amount-match control raises a finding if and only if

$$100\,|a_{\text{inv}} - a_{\text{po}}| > \hat{\theta}_{2.1.1}\, a_{\text{po}}, \tag{2}$$

where a_{inv} and a_{po} are the invoice and PO amounts in integer paise, so rounding cannot change a decision at the boundary, and $\hat{\theta}_{2.1.1}$ is the tolerance in percent read from

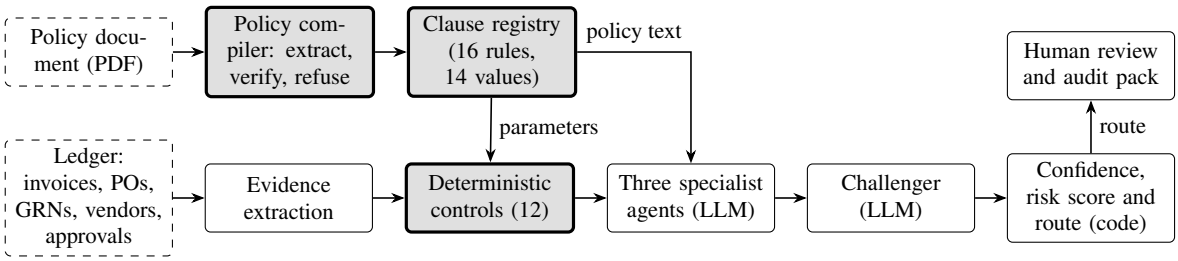


Fig. 1. The proposed system. Grey boxes with a thick border are the added feature, white boxes the base pipeline and dashed boxes data. The controls run before any LLM; scoring and routing are code.

Algorithm 1 Policy compilation with refusal

Input: policy PDF D; reference policy $P_{\text{ref}} = (K, s, \kappa, \theta_{\text{ref}})$
Output: registry $\hat{P}$ or a list Π of problems

$\Lambda \leftarrow$ text lines of D with font size, position, page
$\mathcal{R} \leftarrow$ rules segmented from Λ by layout; $\Pi \leftarrow \emptyset$
for all $k \in K$ **do**
  if $k \notin \mathcal{R}$ **then** $\Pi \leftarrow \Pi \cup \{$"rule k absent"$\}$
  else if k repeated **then** $\Pi \leftarrow \Pi \cup \{$"$k$ ambiguous"$\}$
  else $\hat{s}(k) \leftarrow$ severity stated in $\mathcal{R}_k$
    for all parameter p of k **do**
      $\hat{\theta}_{k,p} \leftarrow$ match the pattern of p in $\mathcal{R}_k$
      if no valid match **then** $\Pi \leftarrow \Pi \cup \{(k, p)\}$
add to Π: rules in $\mathcal{R} \setminus K$; no version; approver role missing
if $\Pi \neq \emptyset$ **then return** Π ▷ refused; nothing stored
return $\hat{P} = (K, \hat{s}, \kappa, \hat{\theta})$ with passages and pages

the document. The approval-limit control takes the limit of the approving role from the registry, not from the approval log, which only records what the approval system believed at the time. The controls produce $F_D(x) = \{c_k(x; \hat{\theta}_k) : k \in K_c\}$.

D. Agentic Layer

The four judgement rules (receipt remarks, description match, near-duplicates and structuring across days) go to three specialist agents whose remits follow the sections of the policy. Prompts contain no thresholds and no rule numbers; the policy text is inserted at run time. In the spirit of multi-agent debate [13], a challenger may remove agent findings that the evidence does not support. In configuration C5, the full pipeline (C1 to C5 are defined in Subsection VI-D), each model finding f receives a confidence

$$c(f) = \tfrac{1}{3}\big(g(f) + p(f) + a(f)\big), \tag{3}$$

where $g = 1$ if every evidence field exists in the transaction with the stated value, $p = 1$ if the cited rule exists in the active policy, and a is the share of $m = 3$ re-runs at temperature 0.7 that cite the same rule. A control finding always has $c(f) = 1$.

E. Risk Scoring and Routing

With weights $w = (30, 20, 10, 5)$ for critical, high, medium and low findings, the count-based and the confidence-weighted scores are

$$R_0(x) = \min\Big(100, \textstyle\sum_{f \in F(x)} w_{s(f)}\Big), \tag{4}$$

$$R_c(x) = \min\Big(100, \textstyle\sum_{f \in F(x)} w_{s(f)}\, c(f)\Big). \tag{5}$$

A critical finding raises the score to at least 70 and a high finding to at least 50; for R_c, only findings with $c(f) \geq \vartheta = 2/3$ trigger these floors. An invoice is escalated if $R_c \geq \tau = 50$, sent to sampled review if $R_c < \tau \leq R_0$, and approved otherwise.

F. Formal Properties

Proposition 1 (Non-Interference)**.** *In every configuration that includes the controls, $F_D(x) \subseteq F(x)$, and each $f \in F_D(x)$ enters (4) and (5) with $c(f) = 1$. Hence $R_c(x) \geq R_0^{\text{C1}}(x)$, and any invoice that the controls alone escalate stays escalated, whatever the LLM stages output.*

Sketch. The challenger can act only on agent findings, and model findings add non-negative terms to (5). The floors set by control findings are not affected. □

Proposition 2 (Drift Identity)**.** *If the controls are faithful under P_0, that is $C(x) = V_{P_0}^{\text{c}}(x)$ for all x, then the frozen auditor satisfies $\text{A}_P(\text{frozen}) = 1 - |\Delta(P_0, P)|/N$.*

Sketch. The frozen auditor cites $V_{P_0}^{\text{c}}(x)$, which equals $V_P^{\text{c}}(x)$ exactly when $x \notin \Delta(P_0, P)$. □

Proposition 3 (Error Polarity)**.** *If a change tightens the policy, $V_{P_0}^{\text{c}}(x) \subseteq V_P^{\text{c}}(x)$ for all x, every error of the frozen auditor is an omission (a missed exception); if it loosens the policy, $V_{P_0}^{\text{c}}(x) \supseteq V_P^{\text{c}}(x)$, every error is a commission (an unsupported finding).*

Sketch. The errors are $V_P^{\text{c}} \setminus V_{P_0}^{\text{c}}$ (omissions) and $V_{P_0}^{\text{c}} \setminus V_P^{\text{c}}$ (commissions), and under either inclusion one of the two sets is empty. For threshold rules the inclusion follows from the order of the parameter: a smaller tolerance, a lower daily count or approval limit, a larger set of non-business days or blocked statuses, or a round-amount multiple that divides the old one all tighten the policy. □

Propositions 2 and 3 make the error of a frozen system a property of the ledger and the change, not of the implementation; Section VII measures it.

VI. Experimental Setup

A. Data

All data are synthetic. As in AuditBench [2], anomalies are injected into synthetic transactions and labelled when injected. An independent validator, sharing no code with the controls, recomputes V_P from the finished tables and rejects any ledger whose labels it cannot reproduce. Each of three

benchmark ledgers of 5,000 invoices contains 420 anomalous invoices (8.4%) in fourteen classes (ten rule-defined, four judgement) and 120 hard negatives, clean invoices built to look like an anomaly. A 20,000-invoice ledger holds 420 anomalous invoices of the same mix (2.1%). Five ledgers with 40,000 invoices in total serve the policy experiments.

B. Policies

From version 2.5 we derived fourteen variants (eleven change one threshold, three change several), twenty more drawn at random with a fixed seed from value ranges never used to build or test the compiler, and sixteen edited copies of the reference document to test refusal. Each is rendered as a PDF file, the only input the system receives.

C. Baselines

We use an isolation forest [6] (300 trees, fitted without labels on all invoices) and a one-class SVM [7] (radial basis function kernel, $\nu = 0.1$, fitted on the clean invoices of the training folds) from scikit-learn [16], on 21 generic features that encode no policy threshold. Thresholds are set by five-fold cross-fitting stratified by class: each fold is decided with the threshold that maximises F1 on the other four, so no invoice is decided with a threshold tuned on its own label. As a detector names no rule, an invoice counts as detected if it is reported at all.

D. LLM Stages

We used Qwen2.5-7B [17] at temperature 0, served locally on a laptop central processing unit (CPU). The configurations are C1, controls alone; C2, one prompt over the whole policy, no controls; C3, controls and three agents; C4, C3 with the challenger; C5, C4 with confidence. Prompts were tuned only on a development ledger and frozen before evaluation (one prompt hash in all runs). We evaluate on 20 development-ledger invoices (one per anomaly class, one per hard-negative type, one ordinary) and on 34 invoices of benchmark ledger 101 (five per judgement class, two per hard-negative type, four ordinary).

E. Statistics

We report 95% percentile bootstrap intervals over invoices (1,000 resamples) and use McNemar's exact test on paired per-invoice correctness, with Holm's adjustment.

VII. Results

A. Compilation Fidelity

The compiler accepted all fourteen variants, recovering 196 of 196 parameter values and the wording of 224 of 224 rules exactly, in 0.01–0.09 s per document; on the twenty held-out variants it recovered 280 of 280 values and 320 of 320 rules. Of the sixteen edited documents, the seven that keep every value were read correctly and the seven that cannot be executed were refused with a reason. The other two were ambiguous (a rule stated twice with different values; an approval table missing a role) and were accepted in a first run. We added both checks with regression tests, after which all sixteen were handled as required.

B. Adherence and Threshold Drift

Table I lists all fourteen variants. The policy-driven controls decided every invoice as the policy in force requires, with no wrong decision in 560,000 audits over the fourteen variants or 800,000 over the twenty held-out ones. Under version 2.5 the controls and the validator agree on all 40,000 invoices, so Proposition 2 applies, and in all 34 variants the errors of the frozen auditor are exactly the displaced decisions. Displacement ranges from zero to 12,274 invoices (30.7%) over the fourteen variants and reaches 7,953 (19.9%, median 79) over the held-out ones. As Proposition 3 predicts, each of the fourteen variants produces errors of one polarity. The five tightening variants cause 24,854 missed exceptions and 24,177 missed escalations, a silent failure; the eight loosening variants cause 1,069 unsupported findings and 913 needless escalations, a cost in reviewer time. The five held-out variants with errors of both polarities each combine a tightening and a loosening part, such as replacing Sunday by Friday as a non-business day.

C. Detection Against Classical Detectors

Table II compares the controls with the detectors. The controls have the highest recall and F1, with a precision of 1.000, and raise no finding on any clean invoice, hard negatives included, while the detectors flag 36% to 56% of the hard negatives across the benchmark ledgers (their false-positive rate, FPR). McNemar's test favours the controls over each detector on every benchmark ledger (Holm-adjusted $p < 10^{-50}$). At the more realistic anomaly rate of the 20,000-invoice ledger, the F1 of the isolation forest drops to 0.211 and that of the one-class SVM to 0.429, while the controls stay at 0.833. The controls' recall of 1.000 on the ten rule-defined classes holds by construction, as they implement the definitions used to inject those anomalies; the detectors have no such advantage. A single binary feature, whether the Goods and Services Tax Identification Number (GSTIN) on the invoice differs from the registered one, separates the two GSTIN classes from every ordinary invoice (on ledger 101, 30 of 30 per class against 0 of 4,460), yet the one-class SVM reports only 3.3% of them on average. Both detectors find the near-duplicates (J1) that the controls leave to the agents, but cannot say which rule is involved.

D. Throughput

The control stage takes 17.8–19.4 μs per invoice in one process, unchanged between ledgers of 5,000 and 40,000 invoices, so its cost is linear in the ledger. Invoices are independent, so the stage runs in parallel without coordination (Fig. 2): on 300,000 invoice audits, throughput rose from 56,660 invoices per second with one worker to 172,226 with eight (speed-up 3.04 on 2 performance and 8 efficiency cores), every configuration producing the same 16,840 findings. Storing every result, finding and trail entry, one process audits 20,000 invoices in under a minute.

E. The LLM Stages

Table III summarises the 34-invoice sample. By design, the controls alone find none of the 20 judgement anomalies. The agents of C3 found 12: all five near-duplicates,

TABLE I. POLICY CHANGE: ADHERENCE TO THE POLICY IN FORCE ON 40,000 INVOICES PER POLICY

	Displaced	Adherence		Frozen: omissions		Frozen: commissions	
Policy variant (new value)	invoices	driven	frozen	exceptions	escalations	findings	escalations
Tolerance 0.5%	62	1.000	0.998	62	62	0	0
Tolerance 2%	7	1.000	1.000	0	0	7	7
Tolerance 5%	63	1.000	0.998	0	0	63	63
Daily count 2	429	1.000	0.989	429	428	0	0
Daily count 5	170	1.000	0.996	0	0	170	170
Manager limit 50,000	7,677	1.000	0.808	7,677	7,514	0	0
Manager limit 150,000	69	1.000	0.998	0	0	69	69
Senior Manager limit 250,000	4,259	1.000	0.894	4,259	4,161	0	0
Round multiple 5,000	0	1.000	1.000	0	0	0	0
Round multiple 50,000	43	1.000	0.999	0	0	43	0
Six-day week	35	1.000	0.999	0	0	35	0
Strict policy	12,274	1.000	0.693	12,427	12,012	0	0
Lenient policy	380	1.000	0.991	0	0	380	302
Version 2.6	302	1.000	0.992	0	0	302	302

Reference version 2.5: tolerance 1%, daily count 3, Manager limit 100,000, Senior Manager limit 500,000, round multiple 10,000, Saturday and Sunday non-business days. Strict: tolerance 0.5%, daily count 2, Manager 50,000, Senior Manager 250,000, round multiple 5,000. Lenient: tolerance 5%, daily count 5, Manager 150,000, six-day week, round multiple 50,000. Version 2.6: tolerance 5%, daily count 5, Manager 150,000. Omissions: required exceptions not raised, and invoices approved that the policy sends to review. Commissions: findings the policy no longer supports, and needless escalations. Exceptions and findings count rules; the other columns count invoices.

TABLE II. INVOICE-LEVEL DETECTION: MEAN OF THREE BENCHMARK LEDGERS (5,000 INVOICES, 8.4% ANOMALOUS)

Measure	Isolation forest	One-class SVM	Controls
Recall	0.577	0.464	0.714
Precision	0.501	0.543	1.000
F1	0.536	0.498	0.833
FPR, hard negatives	0.403	0.481	0.000
FPR, ordinary invoices	0.044	0.025	0.000
F1 on the 20,000-invoice ledger (2.1% anomalous)	0.211	0.429	0.833

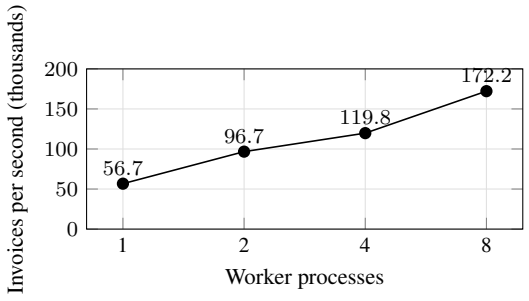


Fig. 2. Throughput of the control stage on one laptop (median of three runs).

four of five goods-receipt remarks, two of five cross-day structuring cases and one of five description mismatches, but only 0.375 of their 32 findings named a rule that was actually violated. The single prompt of C2 found 4, flagged seven of ten hard negatives and all four ordinary invoices, and named a violated rule in 0.031 of its 128 findings. The challenger of C4 halved the false alarms on hard negatives (eight to four) and raised the share of findings naming a violated rule to 0.471, but also removed four of the five correct near-duplicate findings. C5 sent four invoices to sampled review, three of them true anomalies. Agreement across

TABLE III. LLM CONFIGURATIONS ON THE 34-INVOICE SAMPLE (QWEN2.5-7B)

Measure	C2	C3	C4	C5
Judgement anomalies (of 20)	4	12	8	8
Hard negatives flagged (of 10)	7	8	4	4
Ordinary invoices flagged (of 4)	4	3	2	2
Share naming a violated rule	0.031	0.375	0.471	0.471
Sent to sampled review	–	–	–	4
Model calls per invoice	1.21	3.00	3.74	5.24

re-runs did not separate right from wrong: of the model findings kept by C5, 6 of 9 wrong and 5 of 8 right ones had full agreement. On the 20-invoice pilot the controls decided every rule-defined class correctly and, as Proposition 1 requires, all ten control findings and all eight control escalations came through C3–C5 unchanged. No difference between configurations is significant (C3 against C2, $p = 0.277$).

VIII. Discussion

An adherence of 1.000 shows that the mechanism works, not that the rules are good: the controls and the validator are two separate implementations of the same written rules, so their agreement means each document was compiled correctly and no control carries a threshold of its own. The telling numbers are those of the frozen auditor. By Proposition 2 its error equals the displacement of the change, and ordinary revisions of a procurement policy moved from none to almost a third of the decisions on these ledgers; by Proposition 3, a frozen system fails silently whenever the policy is tightened.

For the twelve decidable rules, the citation problem reported by AuditBench cannot occur, since each finding is tied to its rule by the control that implements it. For the four judgement rules, the low share of agent findings naming a violated rule shows the same problem, and neither the challenger nor agreement-based confidence removes it, which

is why those rules stay under human review. The architecture does not make LLMs reliable; it limits where their mistakes can matter, taking every decidable rule out of the model's hands (Proposition 1) and leaving the remaining errors where a reviewer can see them.

IX. Limitations

All ledgers are synthetic, so the counts describe how these ledgers respond to these changes, not the books of a real organisation. The controls and the ground truth implement the same written rules, so perfect scores on the rule-defined classes verify the implementation, not that the rules catch real fraud. The LLM stages were measured on 20 and 34 invoices with one 7B model on a laptop CPU; these samples are too small to separate the configurations, and accuracy on complete benchmark ledgers is unmeasured. Pattern extraction reads one template's wording and refuses other documents with reasons. A model-assisted path for documents in their own wording proposed 80 of 84 values correctly on six short documents, but two wrong values passed its code check, so a named person must confirm values before adoption. All services ran on one laptop; a multi-machine deployment has not been measured.

X. Conclusion and Future Work

We presented a multi-agent platform for forensic procurement audit in which the organisation's policy document is compiled into the thresholds of deterministic controls that run before, and cannot be overridden by, any LLM. On synthetic data the compiler recovered every threshold of fourteen hand-written and twenty held-out documents, the controls made no wrong decision in 1,360,000 invoice audits, and a frozen auditor got up to 30.7% of invoices wrong, as the drift identity predicts. The controls reached an F1 of 0.833 with no false alarm on a clean invoice, against 0.536 and 0.498 for the classical detectors, at 172,226 invoices per second on one laptop. Future work will evaluate the LLM stages on complete ledgers with a larger model, deploy the services across machines, and test the platform on real, anonymised procurement data. The platform, datasets, policy documents and the scripts behind every number are available at `https://github.com/Chandana909/major.project`.

Acknowledgment

The authors thank the Department of Information Technology, Chaitanya Bharathi Institute of Technology, Hyderabad, for its facilities and support.

References

[1] M. A. Vasarhelyi and F. B. Halper, "The Continuous Audit of Online Systems," *Auditing: A Journal of Practice and Theory*, vol. 10, no. 1, pp. 110–125, 1991.

[2] R. Wang, J. Liu, W. Zhao, S. Li, and D. Zhang, "AuditBench: A Benchmark for Large Language Models in Financial Statement Auditing," in *AI for Research and Scalable, Efficient Systems*, Communications in Computer and Information Science. Singapore: Springer, 2025, pp. 59–81.

[3] Q. Liu *et al.*, "Large Language Model-Based Multi-Agent Systems for Financial Markets Simulation: A Survey," *Science China Information Sciences*, vol. 69, no. 7, Art. no. 171201, 2026.

[4] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[5] R. J. Bolton and D. J. Hand, "Statistical Fraud Detection: A Review," *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002.

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[7] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the Support of a High-Dimensional Distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.

[8] L. Hillebrand *et al.*, "Improving Zero-Shot Text Matching for Financial Auditing with Large Language Models," in *Proc. ACM Symp. Document Engineering (DocEng)*, 2023, pp. 1–4.

[9] A. Berger *et al.*, "Towards Automated Regulatory Compliance Verification in Financial Auditing with Large Language Models," in *Proc. IEEE Int. Conf. Big Data*, 2023, pp. 4626–4635.

[10] Z. Ji *et al.*, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[11] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[12] T. Guo *et al.*, "Large Language Model Based Multi-Agents: A Survey of Progress and Challenges," in *Proc. 33rd Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2024, pp. 8048–8057.

[13] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving Factuality and Reasoning in Language Models Through Multiagent Debate," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, PMLR vol. 235, 2024.

[14] E. Mosqueira-Rey, E. Hernández-Pereira, D. Alonso-Ríos, J. Bobes-Bascarán, and Á. Fernández-Leal, "Human-in-the-Loop Machine Learning: A State of the Art," *Artificial Intelligence Review*, vol. 56, no. 4, pp. 3005–3054, 2023.

[15] LangChain, "LangGraph," software. [Online]. Available: `https://github.com/langchain-ai/langgraph`

[16] F. Pedregosa *et al.*, "Scikit-Learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[17] Qwen Team, "Qwen2.5 Technical Report," arXiv:2412.15115, 2024.